

DEVOPS INTERVIEW PLAYBOOK

25 Real DevOps Interview Scenarios

Scenario-based questions, production-grade answers, and field-tested tips

Compiled by Govind

Real scenarios · AKS · Kubernetes · Terraform · Linux · CI/CD · Azure Networking

Contents

01	AKS	You have 20 microservices running in an AKS cluster. Will you create one LoadBalancer Service for each application?	4
02	AZURE NETWORKING	Your AKS cluster is private, but internet users should still be able to access your application. How would you design the traffic flow?	5
03	TERRAFORM	Two DevOps engineers accidentally ran terraform apply at the same time. How would you prevent state corruption?	6
04	LINUX	Production server CPU suddenly becomes 100%. What's your first approach?	7
05	CI/CD	Deployment completed successfully, but production is still showing the old version. Where do you start debugging?	8
06	KUBERNETES	Your Pod is in Pending state for the last 30 minutes. How will you troubleshoot it?	9
07	KUBERNETES	One of your applications suddenly starts showing CrashLoopBackOff. Where do you start debugging?	10
08	KUBERNETES	Your application cannot connect to the database after deployment. How will you troubleshoot it?	11
09	AZURE AKS	Your AKS cluster has both system and user node pools. Why not deploy everything on a single node pool?	12
10	CI/CD	A deployment failed in production halfway through. Some Pods are running the new version while others are still on the old version. What will you do?	13
11	KUBERNETES	Your application is running, but users are getting 502 Bad Gateway. How will you troubleshoot it?	14
12	AZURE NETWORKING	Your application should access Azure Storage privately. You don't want traffic going over the public internet. How would you design it?	15
13	TERRAFORM	Someone manually changed an Azure resource from the Portal. How will Terraform detect it?	16
14	LINUX	Your production server suddenly runs out of disk space. What's your debugging approach?	17
15	CI/CD	A developer says, "The pipeline is failing only in production. Dev and QA work perfectly." How will you troubleshoot it?	18
16	KUBERNETES	A Deployment is successful, all Pods are Running, but users still can't access the application. Where will you start debugging?	19
17	AZURE NETWORKING	Your AKS cluster needs internet access to pull Docker images, but worker nodes shouldn't have Public IPs. How would you design it?	20
18	TERRAFORM	Your Terraform deployment accidentally deleted an Azure resource. How would you prevent this in production?	21
19	LINUX	A production application suddenly starts failing with 'Too many open files'. How will you troubleshoot it?	22

20	CI/CD	Your deployment pipeline completed successfully, but one microservice wasn't updated while all others were. How will you troubleshoot it?	23
21	KUBERNETES	Your application needs to run on only 2 specific worker nodes. How will you achieve this?	24
22	AZURE	Your company wants to connect 15 VNets across different Azure regions. Will you create VNet Peering between every VNet?	25
23	TERRAFORM	Your company already has hundreds of Azure resources created manually. Now they want to manage everything using Terraform. Will you recreate all resources?	26
24	LINUX	Users report that the application is slow, but CPU and Memory usage look completely normal. How will you investigate?	27
25	CI/CD	A developer accidentally pushed directly to the production branch. How would you prevent this from happening again?	28

01

AKS

You have 20 microservices running in an AKS cluster. Will you create one LoadBalancer Service for each application?

INTERVIEW ANSWER

No.

I would avoid creating a separate Azure Load Balancer for every service because that increases cost and management overhead.

Instead, I'd expose only the Ingress Controller using a single LoadBalancer Service. Then I'd use Ingress rules to route traffic to multiple backend services based on hostname or path.

For example:

- api.company.com → API Service
- auth.company.com → Auth Service
- www.company.com → Frontend

This is the approach I've seen in production because it's scalable and much easier to manage.

WHY?

Azure creates a Public IP and Load Balancer for every Service of type LoadBalancer.

If you have 20 services:

- 20 Public IPs
- Higher cost
- More difficult DNS management

Using NGINX Ingress or Application Gateway allows one entry point for all applications.

PRODUCTION TIP

In production, I rarely expose application Services directly.

Instead, I expose only the Ingress Controller.

02

AZURE NETWORKING

Your AKS cluster is private, but internet users should still be able to access your application. How would you design the traffic flow?

INTERVIEW ANSWER

Since the worker nodes are private, I wouldn't expose them directly.

The incoming request would first reach Azure Application Gateway (or Azure Load Balancer depending on the architecture).

Application Gateway forwards the request to the Ingress Controller.

The Ingress Controller routes it to the appropriate Kubernetes Service, which finally forwards it to the Pods.

The worker nodes remain private throughout the process.

REQUEST FLOW

Internet

↓

Application Gateway

↓

Ingress Controller

↓

Service

↓

Pods

PRODUCTION TIP

Never expose worker nodes directly.

03

TERRAFORM

Two DevOps engineers accidentally ran terraform apply at the same time. How would you prevent state corruption?

INTERVIEW ANSWER

I wouldn't store the Terraform state locally.

I'd configure a remote backend such as Azure Storage Account with state locking.

This ensures only one person can modify the infrastructure at a time.

If another engineer starts terraform apply, Terraform waits until the lock is released.

PRODUCTION TIP

Never use local state for production.

04

LINUX

Production server CPU suddenly becomes 100%. What's your first approach?

INTERVIEW ANSWER

I don't immediately restart the server.

First, I identify which process is consuming the CPU.

My typical flow is:

- top or htop
- ps aux --sort=-%cpu
- pidstat
- sar
- journalctl

Once I identify the process, I investigate why it's consuming resources before taking any action.

PRODUCTION TIP

Never restart a production server without identifying the root cause.

05

CI/CD

Deployment completed successfully, but production is still showing the old version. Where do you start debugging?

INTERVIEW ANSWER

I debug layer by layer instead of assuming the pipeline is correct.

My approach is:

1. Verify the pipeline actually pushed the latest image.
2. Check whether Kubernetes Deployment was updated.
3. Verify the image tag.
4. Confirm Pods restarted.
5. Check Service and Ingress.
6. Clear CDN/browser cache if applicable.

This systematic approach helps isolate the issue quickly.

PRODUCTION TIP

A green pipeline doesn't always mean the latest application is running.

06

KUBERNETES

Your Pod is in Pending state for the last 30 minutes. How will you troubleshoot it?

INTERVIEW ANSWER

I won't immediately assume the cluster is broken.

First, I'll identify why the scheduler isn't placing the Pod on any node.

My debugging flow would be:

1. Check the Pod events using `kubectl describe pod`.
2. Verify if there are enough resources on worker nodes.
3. Check NodeSelector, Node Affinity, Taints & Tolerations.
4. Verify PVC binding if the Pod uses storage.
5. Check whether the Cluster Autoscaler is working.

Once I know why the scheduler rejected the Pod, I'll fix that specific issue instead of randomly restarting components.

WHY?

A Pod remains in Pending only when Kubernetes cannot schedule it.

Common reasons include:

- Insufficient CPU/Memory
- Taints
- Node Affinity mismatch
- PVC Pending
- No available nodes

PRODUCTION TIP

Always start with:

```
kubectl describe pod
```

The Events section usually tells you why the scheduler rejected the Pod.

07

KUBERNETES

One of your applications suddenly starts showing CrashLoopBackOff. Where do you start debugging?

INTERVIEW ANSWER

CrashLoopBackOff simply means the container keeps crashing after starting.

I don't restart the Deployment immediately.

Instead, I follow this sequence:

- Check current logs
- Check previous logs
- Describe the Pod
- Verify environment variables
- Check Secrets and ConfigMaps
- Verify application startup

Only after identifying the root cause do I redeploy.

WHY?

CrashLoopBackOff is usually caused by:

- Wrong environment variables
- Missing Secrets
- Database connection failure
- Application startup exception
- Wrong image

PRODUCTION TIP

Always check:

```
kubectl logs --previous
```

Many engineers only check current logs and miss the actual crash.

08

KUBERNETES

Your application cannot connect to the database after deployment. How will you troubleshoot it?

INTERVIEW ANSWER

I debug layer by layer instead of assuming it's a database issue.

My approach is:

1. Verify the Pod is running.
2. Check the application logs.
3. Verify Secret values.
4. Check Service DNS.
5. Test connectivity from inside the Pod.
6. Verify NetworkPolicy.
7. Finally check database firewall rules.

This helps isolate whether the issue is application, Kubernetes networking, or infrastructure.

WHY?

In production, database issues are often caused by:

- Wrong Secret
- Incorrect Service name
- DNS resolution failure
- NetworkPolicy
- Firewall restrictions

Not by the database itself.

PRODUCTION TIP

One command I always use:

```
kubectl exec -it -- sh
```

Then test connectivity from inside the container.

09

AZURE AKS

Your AKS cluster has both system and user node pools. Why not deploy everything on a single node pool?

INTERVIEW ANSWER

I prefer separating workloads.

The system node pool is reserved for Kubernetes components like CoreDNS, Metrics Server, and CNI.

Application workloads run on user node pools.

This ensures system components always have dedicated resources and application traffic doesn't impact cluster stability.

WHY?

Benefits include:

- Better resource isolation
- Easier upgrades
- Independent scaling
- Improved reliability

PRODUCTION TIP

In production, I always use multiple node pools for different workloads such as:

- System
- General applications
- Spot workloads
- GPU workloads

10

CI/CD

A deployment failed in production halfway through. Some Pods are running the new version while others are still on the old version. What will you do?

INTERVIEW ANSWER

First, I'll stop the rollout from progressing further.

Then I'll check the Deployment rollout status and identify why new Pods aren't becoming Ready.

If the issue affects production traffic, I'll immediately roll back to the previous stable revision.

Once the application is stable, I'll investigate the root cause before attempting another deployment.

WHY?

Rolling back quickly minimizes production impact.

Typical reasons include:

- Readiness probe failure
- Image issue
- Configuration change
- Secret mismatch
- Resource limits

PRODUCTION TIP

First command I run:

```
kubectl rollout status deployment
```

If required:

```
kubectl rollout undo deployment
```

Never keep retrying deployments in production without understanding why the rollout failed.

11

KUBERNETES

Your application is running, but users are getting 502 Bad Gateway. How will you troubleshoot it?

INTERVIEW ANSWER

I don't assume it's an Ingress issue immediately.

I debug the request flow layer by layer.

My approach would be:

1. Check whether the Pods are Running and Ready.
2. Verify the Service endpoints.
3. Check whether the Service selector matches the Pods.
4. Inspect the Ingress configuration.
5. Review Ingress Controller logs.
6. Finally, test the application inside the Pod.

This helps me identify exactly where the request is failing instead of guessing.

WHY?

A 502 Bad Gateway can happen because:

- Backend Pods aren't Ready
- Service has no endpoints
- Wrong Service port
- Wrong Ingress backend
- Application isn't listening on the expected port

PRODUCTION TIP

One of the first commands I run is:

```
kubectl get endpoints
```

If there are 0 endpoints, I immediately know the Service has nothing to forward traffic to.

12

AZURE NETWORKING

Your application should access Azure Storage privately. You don't want traffic going over the public internet. How would you design it?

INTERVIEW ANSWER

I would use a Private Endpoint.

This assigns a private IP from the VNet to the Storage Account.

Now the application communicates over Microsoft's private network instead of the internet.

I'd also configure Private DNS so the application resolves the Storage Account to the private IP.

WHY?

Benefits:

- No public exposure
- Better security
- Lower attack surface
- Compliance requirements

PRODUCTION TIP

Private Endpoint alone isn't enough.

Private DNS configuration is equally important.

13

TERRAFORM

Someone manually changed an Azure resource from the Portal.
How will Terraform detect it?

INTERVIEW ANSWER

This is called configuration drift.

The first thing I'd do is run:

```
terraform plan
```

Terraform compares the current infrastructure with the state file and configuration.

If someone modified the resource manually, Terraform will show the difference before making any changes.

Then I'll decide whether to:

- Keep the manual change
- Revert it
- Update the Terraform code

WHY?

Infrastructure should have a single source of truth.

Manual changes create inconsistencies that Terraform will eventually detect.

PRODUCTION TIP

Avoid making production changes directly from the Azure Portal.
Everything should go through Terraform.

14

LINUX

Your production server suddenly runs out of disk space. What's your debugging approach?

INTERVIEW ANSWER

I won't immediately delete files.

First, I'll identify what's consuming the disk.

My approach is:

1. Check filesystem usage.
2. Find which directory is growing.
3. Identify the largest files.
4. Check logs.
5. Verify Docker images or container logs if it's a Kubernetes worker.

Only after understanding the cause would I clean up unnecessary data.

WHY?

Common causes include:

- Log files
- Docker images
- Container logs
- Backup files
- Application-generated files

PRODUCTION TIP

The first command I usually run is:

```
df -h
```

Then:

```
du -sh /* | sort -h
```

to quickly locate the biggest directories.

15

CI/CD

A developer says, “The pipeline is failing only in production. Dev and QA work perfectly.” How will you troubleshoot it?

INTERVIEW ANSWER

I won't assume the pipeline is the problem.

Since Dev and QA are working, I'd compare what's different in Production.

My checklist would be:

- Environment variables
- Secrets
- Service Connections
- Deployment approvals
- Resource permissions
- Kubernetes manifests
- Production-specific configurations

Usually, the issue is caused by environment-specific differences rather than the pipeline itself.

WHY?

Most production-only failures happen because Production has:

- Different secrets
- Different infrastructure
- Additional approvals
- RBAC restrictions
- Different configuration values

PRODUCTION TIP

Whenever only one environment fails, don't compare the code first.
Compare the environment. That's where the root cause is most often found.

16

KUBERNETES

A Deployment is successful, all Pods are Running, but users still can't access the application. Where will you start debugging?

INTERVIEW ANSWER

I won't assume the application is healthy just because the Pods are running.

I'll trace the complete request path:

1. Verify Pods are Ready, not just Running.
2. Check the Service selector and endpoints.
3. Test the Service internally.
4. Verify the Ingress configuration.
5. Check the Ingress Controller logs.
6. Finally, verify DNS resolution.

I always isolate which layer is breaking the request before fixing anything.

WHY?

A Running Pod doesn't mean traffic can reach it.

The issue could be:

- Readiness Probe
- Service selector mismatch
- Missing Endpoints
- Incorrect Ingress rules
- DNS issue

PRODUCTION TIP

Always think:

Client → DNS → Ingress → Service → Pod

17

AZURE NETWORKING

Your AKS cluster needs internet access to pull Docker images, but worker nodes shouldn't have Public IPs. How would you design it?

INTERVIEW ANSWER

I wouldn't assign Public IPs to worker nodes.

Instead, I'd deploy the cluster in a private subnet and configure a NAT Gateway for outbound internet access.

This allows the nodes to pull images, reach Azure services, and call external APIs while remaining private.

WHY?

Benefits:

- No Public IP on nodes
- Secure outbound connectivity
- Predictable outbound IP
- Better security posture

PRODUCTION TIP

Private AKS + NAT Gateway is a common production architecture.

18

TERRAFORM

Your Terraform deployment accidentally deleted an Azure resource. How would you prevent this in production?

INTERVIEW ANSWER

In production, I don't rely only on terraform apply.

I implement multiple safety checks:

- Mandatory terraform plan review
- Pull Request approval
- Protected branches
- Remote backend
- Resource protection using `prevent_destroy`
- Separate environments

These controls reduce the risk of accidental deletions.

WHY?

Human mistakes happen.

Good Terraform practices are about preventing destructive changes before they reach production.

PRODUCTION TIP

For critical resources like production databases or Key Vaults, I use:

```
lifecycle {
  prevent_destroy = true
}
```

This forces Terraform to stop before deleting the resource.

19

LINUX

A production application suddenly starts failing with 'Too many open files'. How will you troubleshoot it?

INTERVIEW ANSWER

This usually indicates the application has exhausted its file descriptor limit.

My approach is:

1. Verify the error in application logs.
2. Check the current file descriptor usage.
3. Identify which process is consuming the descriptors.
4. Verify system limits.
5. Fix the application if it's leaking file handles instead of only increasing the limit.

WHY?

Increasing the limit without finding the cause only hides the real problem.

Common causes include:

- File descriptor leaks
- Too many network connections
- Log files left open
- Application bugs

PRODUCTION TIP

The first command I usually run is:

```
ulimit -n
```

Then inspect the process using:

```
lsof
```

20

CI/CD

Your deployment pipeline completed successfully, but one microservice wasn't updated while all others were. How will you troubleshoot it?

INTERVIEW ANSWER

Since only one service is affected, I wouldn't investigate the entire pipeline first.

I'd isolate that specific service.

My checklist would be:

1. Verify whether a new Docker image was built.
2. Check if the image was pushed successfully.
3. Verify the Deployment is using the latest image tag.
4. Check rollout status.
5. Verify Pods restarted.
6. Review application logs.

This systematic approach helps determine whether the issue is in CI, CD, Kubernetes, or the application itself.

WHY?

A single-service deployment failure is often caused by:

- Incorrect image tag
- Cached image
- Manifest mismatch
- Failed rollout
- Deployment skipped in the pipeline

PRODUCTION TIP

I always compare the deployed image with the image generated by the pipeline before making any changes. If they don't match, I know the problem lies between the CI pipeline and the Kubernetes deployment, not in the application itself.

21

KUBERNETES

Your application needs to run on only 2 specific worker nodes. How will you achieve this?

INTERVIEW ANSWER

It depends on the requirement.

If I simply want the Pods to prefer certain nodes, I'd use Node Affinity.

If they must run only on specific nodes, I'd label those nodes and use `requiredDuringSchedulingIgnoredDuringExecution`.

If the nodes are dedicated for a specific workload, I'd also use Taints & Tolerations so no other applications can be scheduled there.

WHY?

Different scheduling requirements need different Kubernetes features.

- Node Selector → Simple matching
- Node Affinity → Flexible scheduling
- Taints & Tolerations → Dedicated nodes

PRODUCTION TIP

For production databases or monitoring workloads, I usually combine Node Affinity with Taints & Tolerations.

22

AZURE

Your company wants to connect 15 VNets across different Azure regions. Will you create VNet Peering between every VNet?

INTERVIEW ANSWER

No.

Creating peering between every VNet doesn't scale well because the number of peerings grows rapidly as VNets increase.

I'd design a Hub-and-Spoke architecture.

The Hub VNet would contain shared services like Azure Firewall, VPN Gateway, Bastion, and DNS, while all application VNets connect only to the Hub.

This is easier to manage, more secure, and scales much better.

WHY?

Hub-and-Spoke provides:

- Centralized security
- Easier routing
- Lower operational complexity
- Better governance

PRODUCTION TIP

Most enterprise Azure environments use Hub-and-Spoke rather than full mesh peering.

23

TERRAFORM

Your company already has hundreds of Azure resources created manually. Now they want to manage everything using Terraform. Will you recreate all resources?

INTERVIEW ANSWER

No.

Recreating production resources could cause downtime.

Instead, I'd import the existing resources into Terraform state using `terraform import`, then gradually organize them into reusable modules.

Once Terraform manages the state, future changes can be done safely through code.

WHY?

Importing avoids:

- Downtime
- Resource recreation
- Data loss

PRODUCTION TIP

Always run a `terraform plan` after importing to ensure the code matches the existing infrastructure before applying any changes.

24

LINUX

Users report that the application is slow, but CPU and Memory usage look completely normal. How will you investigate?

INTERVIEW ANSWER

High CPU isn't the only reason applications become slow.

I'd investigate other possible bottlenecks like:

- Disk I/O
- Network latency
- DNS resolution
- Database response time
- Application logs

Performance troubleshooting should always be end-to-end rather than focusing only on CPU and memory.

WHY?

Application latency can come from many sources:

- Slow storage
- External API delays
- Network congestion
- Database locks
- DNS issues

PRODUCTION TIP

Never conclude “the server is healthy” just because CPU usage is low.

25

CI/CD

A developer accidentally pushed directly to the production branch. How would you prevent this from happening again?

INTERVIEW ANSWER

I wouldn't rely on developers remembering the process.

Instead, I'd enforce controls in the repository.

My approach would include:

- Protected branches
- Pull Request approvals
- Required code reviews
- Status checks before merge
- Restrict direct pushes
- Automated CI validation

This ensures production changes follow a controlled review process.

WHY?

Good CI/CD isn't just about automation—it's also about governance and reducing human error.

PRODUCTION TIP

In production, prevention is always better than recovery. A properly configured branch protection policy can stop accidental production deployments before they ever reach the pipeline.

That's a wrap.

25 real scenarios, straight from production war rooms.

Compiled by Govind — follow for more DevOps content.